

ARCHITECTURE & IMPLEMENTATION

Hotel Management System

Layered Architecture



Multi-role Platform



Secure REST API



Modular Design (C4)



Executive Summary

SYSTEM OVERVIEW



Monolithic Flask Architecture

Robust backend system providing JSON REST APIs, operated by a single server for easy deployment and maintenance.



Strict Layered Design

Clear separation of concerns: Controller (HTTP) → Service (Business Logic) → Repository (Data Access) → Model.



Quality & Security (NFR)

Compliance with requirements for modifiability, maintainability, security (RBAC), and horizontal scalability via stateless design.

TECHNOLOGY

Python Flask + MySQL



Booking



Payment & Wallet



Services & Staff



Reports

INTERFACE

Multi-role User Interface (Web)

System Overview

ROLES AND FUNCTIONAL SCOPE



User

End Customer (Website)

- Search and book rooms
- Make secure online payments
- Manage digital wallet
- Request additional services
- View booking history



Receptionist

Daily Operations

- Manage check-in/check-out
- Process walk-in bookings
- Assign rooms to guests
- Track transactions and payments
- Manage ongoing service requests



Administrator

Configuration & Management

- Manage room inventory
- Manage user and staff accounts
- Configure hotel services
- Access analytics and reports
- Manage promotional coupons

Functional Requirements

USER (FR-01 TO FR-08)



ID	REQUIREMENT NAME	DESCRIPTION	PRIORITY
FR-01	Search & Booking	Search for rooms by type/status and complete booking to confirm availability.	High
FR-02	Authentication	Secure registration and login to access protected features.	High
FR-03	Booking Management	View booking history and cancel existing bookings.	High
FR-04	Online Payment	Make secure payments for bookings and additional services.	High
FR-05	Service Request	Book additional services (spa, meals, etc.) during stay.	Medium
FR-06	E-Wallet	Check balance, top-up wallet, and pay using internal balance.	High
FR-07	Apply Coupon	Apply valid promotional codes for discounts.	Medium

Functional Requirements

RECEPTIONIST & ADMINISTRATOR (FR-09 TO FR-20)



Receptionist		
FR-09	Check-in/Check-out Process guest arrivals and departures with booking validation.	High
FR-10	Walk-in Booking Create bookings for arriving customers without prior reservation.	High
FR-11	Room Assignment Allocate rooms based on availability and customer preferences.	High
FR-12	Activity History View history of transactions and system actions.	High
FR-13	Service Management Track and update the status of service requests (spa, etc.).	High
FR-14	Refund Automatic wallet refund process in case of cancellations.	High

Administrator		
FR-16	Room Catalog Perform full CRUD operations on room catalog (add, edit, status).	High
FR-17	Staff Management Create and manage staff accounts and roles.	High
FR-18	User Management Overview and management of user accounts.	High
FR-19	Manage Services Update hotel services catalog (prices, availability).	Medium
FR-20	Analytics & Reports Generate reports on occupancy rates, revenue, and performance.	Medium
Note Administrator also has full receptionist privileges.		

NFR & ASR

QUALITY AND ARCHITECTURAL REQUIREMENTS



QUALITY ATTRIBUTES

NFR (Non-Functional)

NFR-01 **Modifiability**
Easy to add new features

NFR-02 **Maintainability**
Code organized by clear responsibilities

NFR-03 **Reliability and Consistency**
Data integrity for bookings/payments

NFR-04 **Security**
Authentication and Authorization (RBAC)

NFR-05 **Usability**
Clear role-based user interfaces



ARCHITECTURAL DECISIONS

ASR (Architecturally Significant)

ASR-1 **Anti-Overbooking**
Overlapping SQL queries in repository

ASR-2 **Financial Security**
RBAC Middleware h user_id validation

ASR-3 **Layered Modifiability**
Controller → Service → Repository (Strict)

ASR-4 **Maintainable Organization**
Files grouped by technical responsibility.

ASR-5 **Error Handling**
Standardized JSON responses (Views)

Use Cases

OVERVIEW OF ACTORS AND INTERACTIONS



User

Web Browser

🔍 View rooms

📅 Booking

💳 Pay for booking

<<include>>

🏷️ Apply discount code

<<extend>>

💰 Top-up wallet

🛎️ Request service

🔄 View/Cancel booking



Receptionist

Front Desk Operations

👤➔ Guest check-in

💳 Guest payment

🚶 Walk-in Booking

🔑 Room Assignment

☰ Transaction history

🔔 Service management



Administrator

Management & Configuration

🏠 Manage rooms (CRUD)

👤 Manage staff

⚙️ Manage hotel services

👥 Manage users

📊 View analytics

🎫 Manage coupons

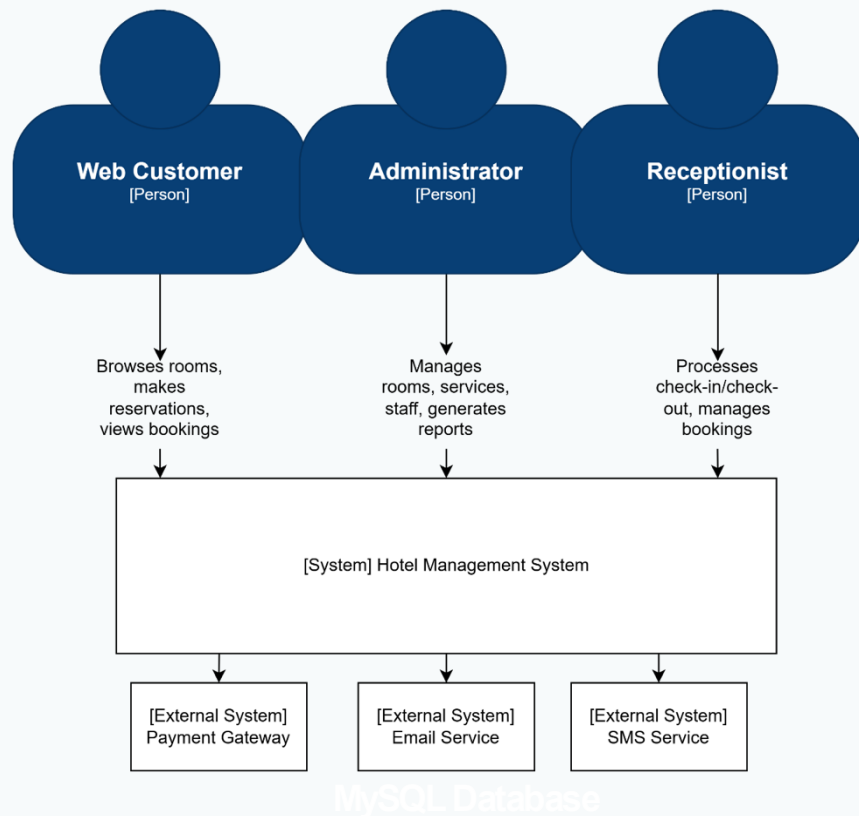
C4 Architecture

LEVEL 1 (CONTEXT) & LEVEL 2 (CONTAINER)



LEVEL 1

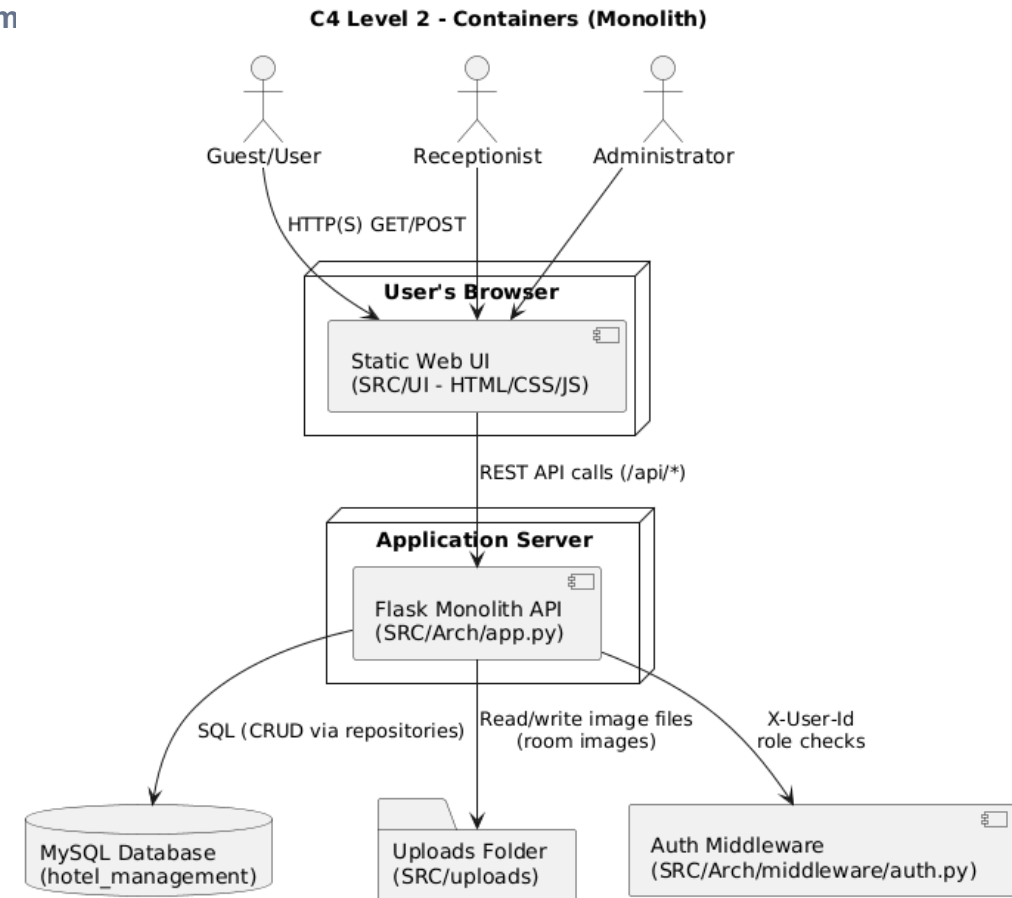
System Context



LEVEL 2

Container Diagram

Hotel Management System

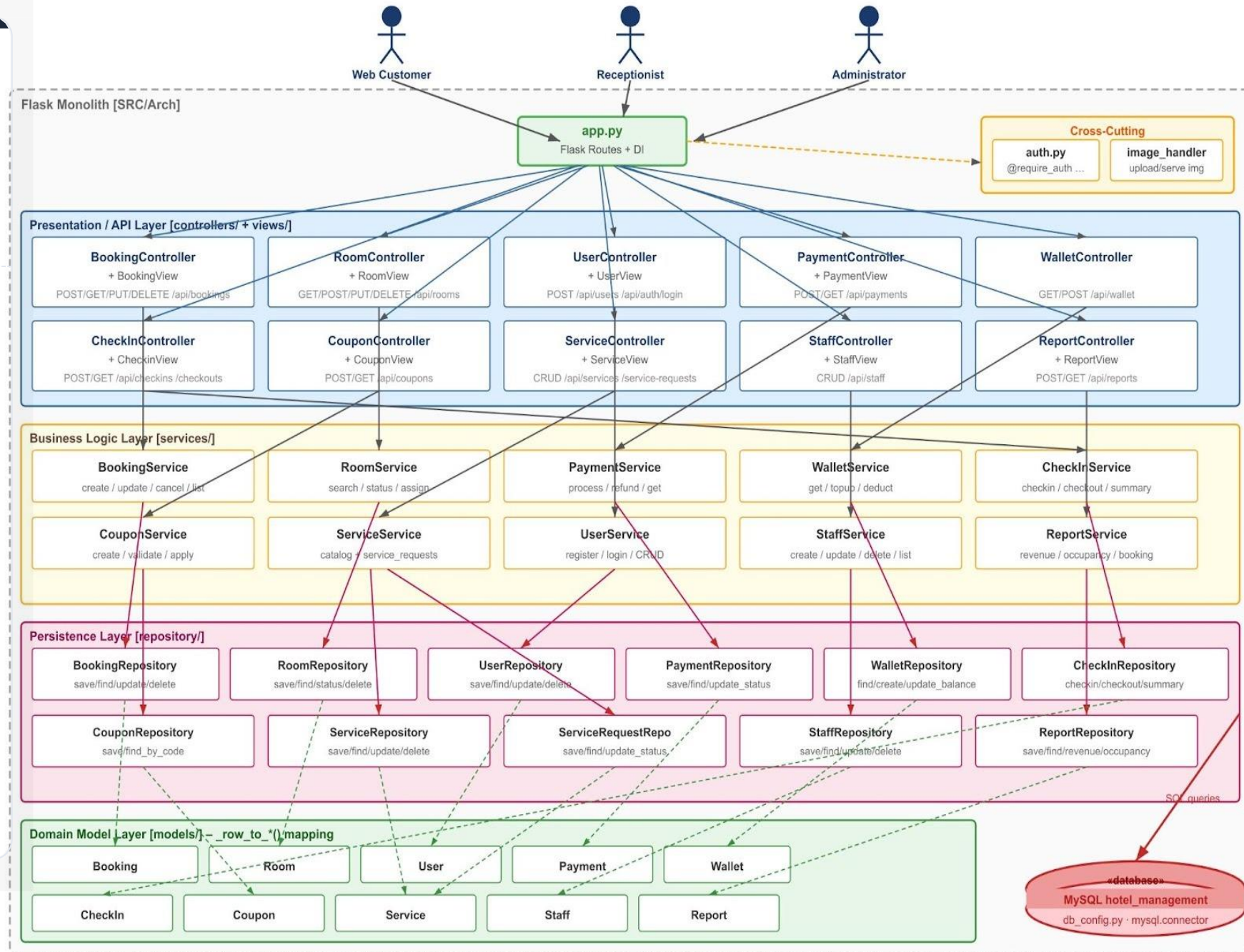




Layered Architecture

C4 LEVEL 3 - COMPONENT DIAGRAM

C4 Level 3 – Component Diagram
Hotel Booking Management System (Flask Monolith – SRC/Arch)



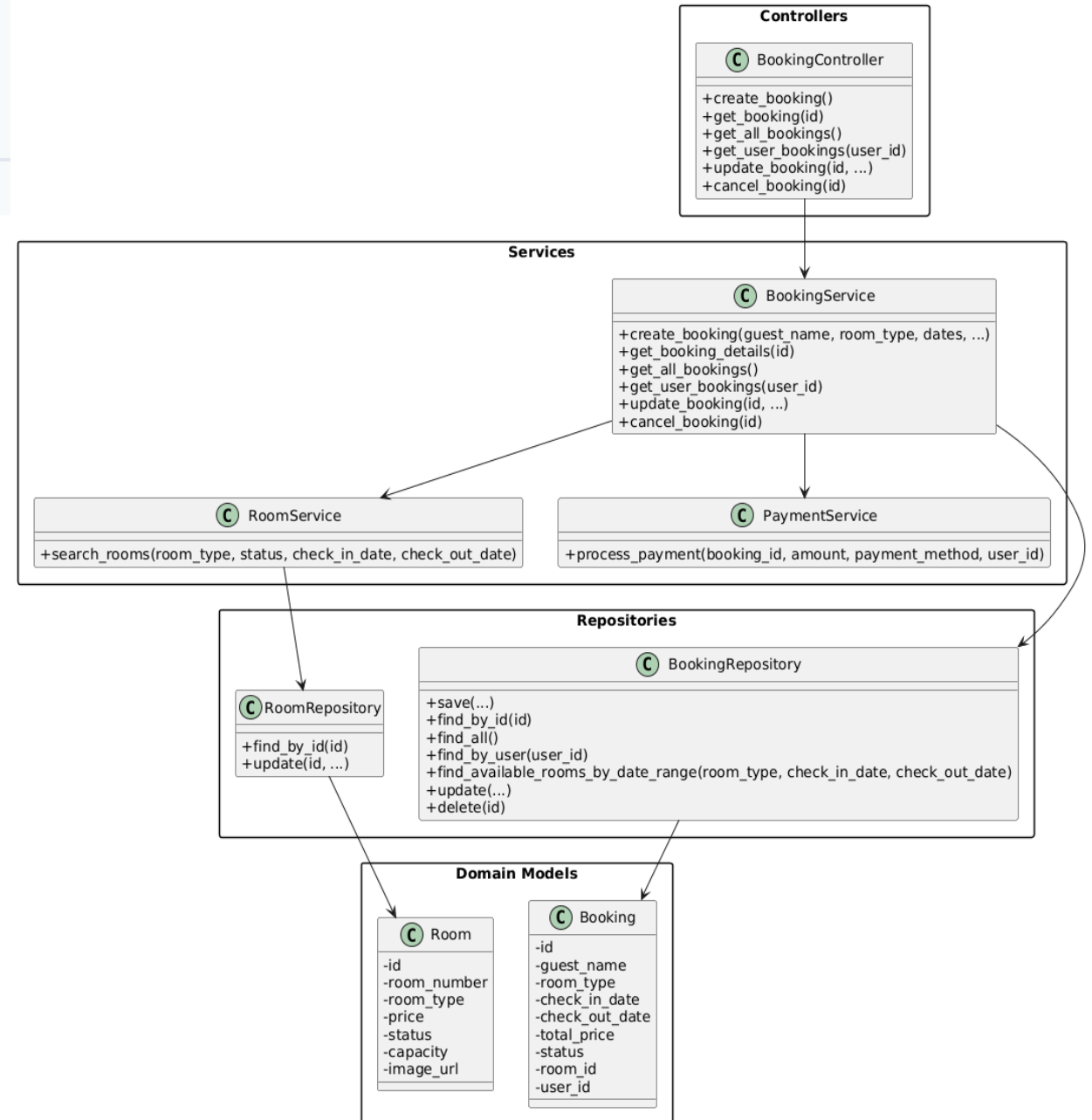
C4 – Code Slices

DETAILED VIEW (DOMAIN SLICES) - LEVEL 4



Booking Domain

C4 Level 4 - Code View (Booking Domain)



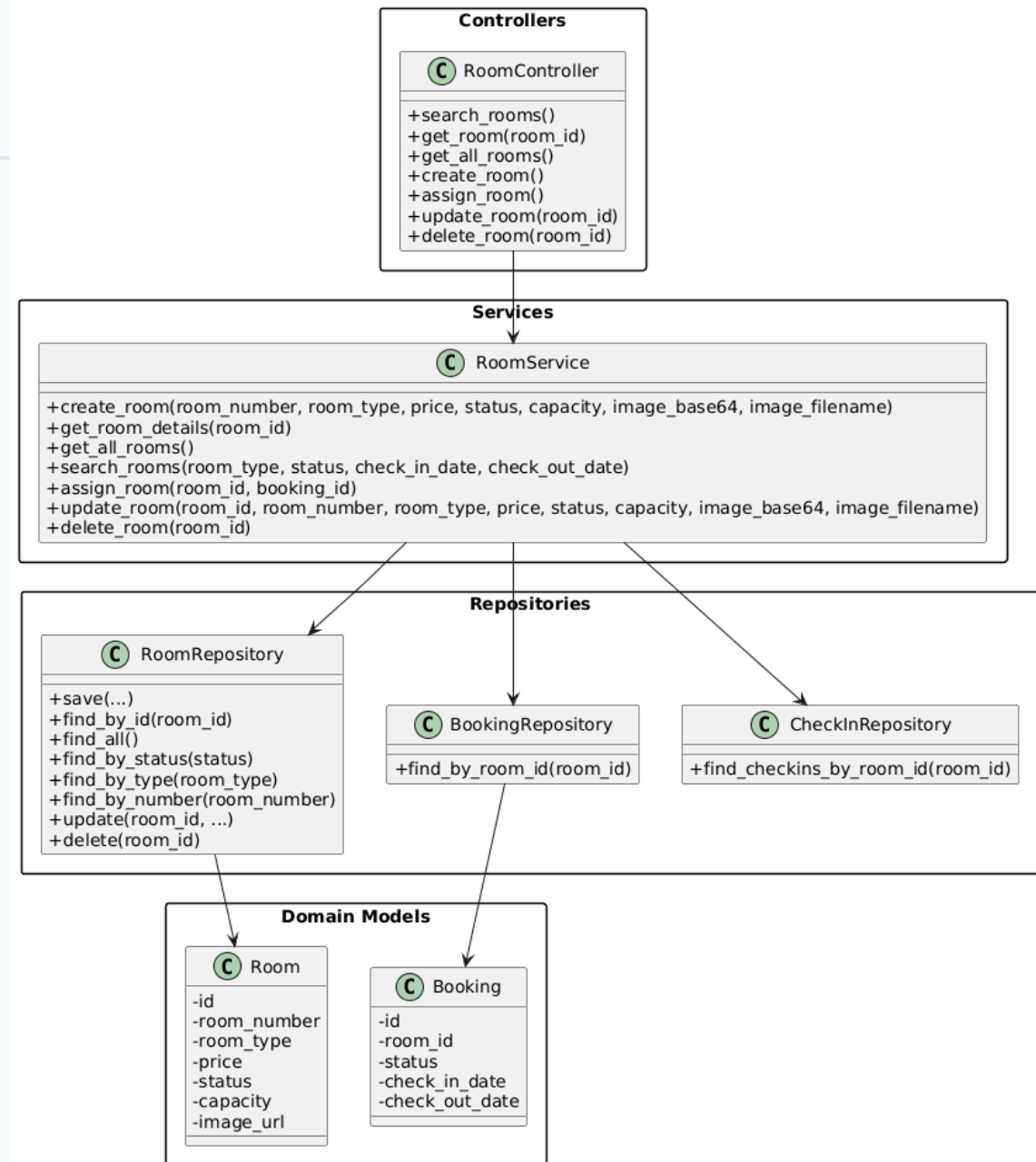
C4 – Code Slices

DETAILED VIEW (DOMAIN SLICES) - LEVEL 4



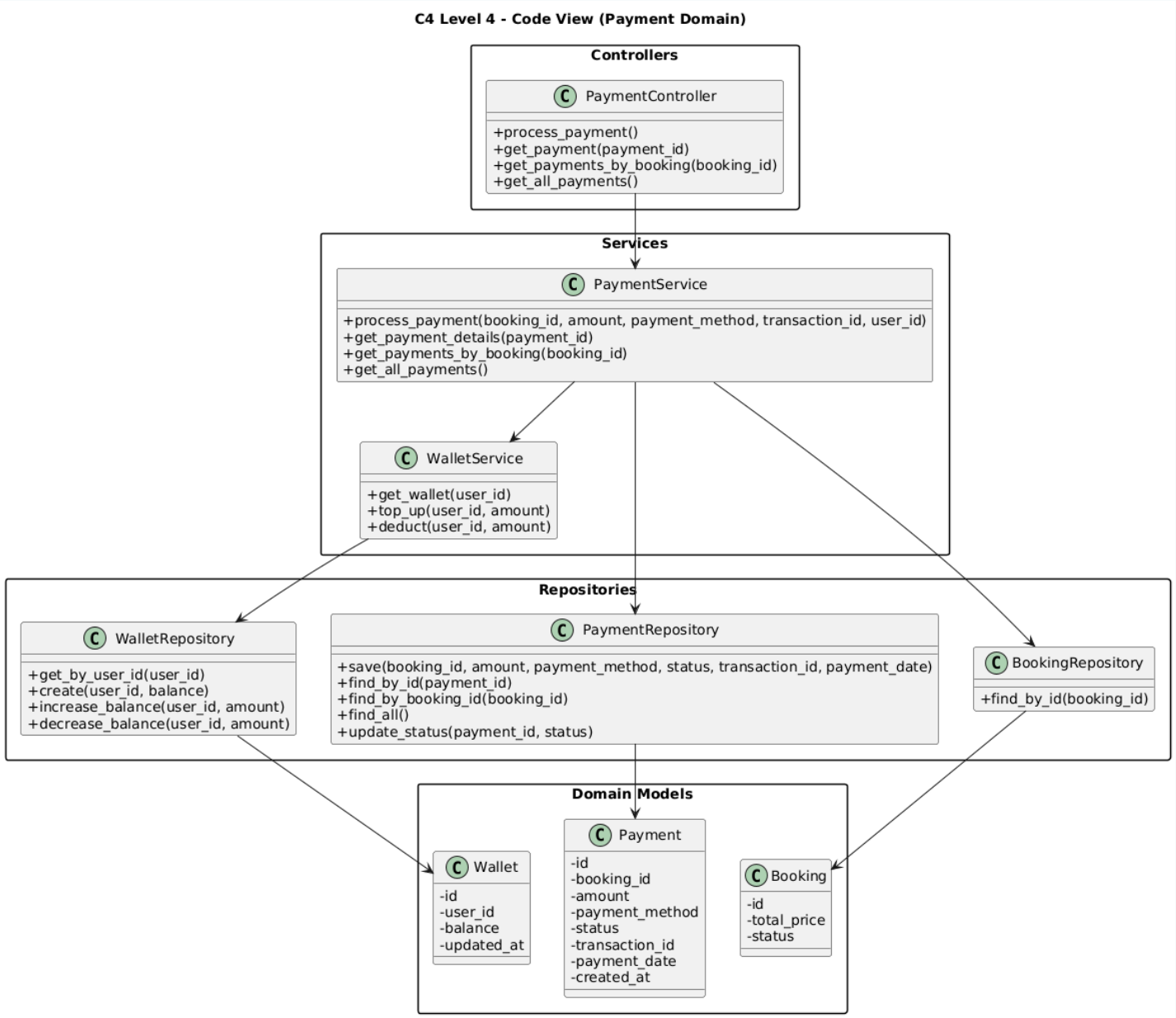
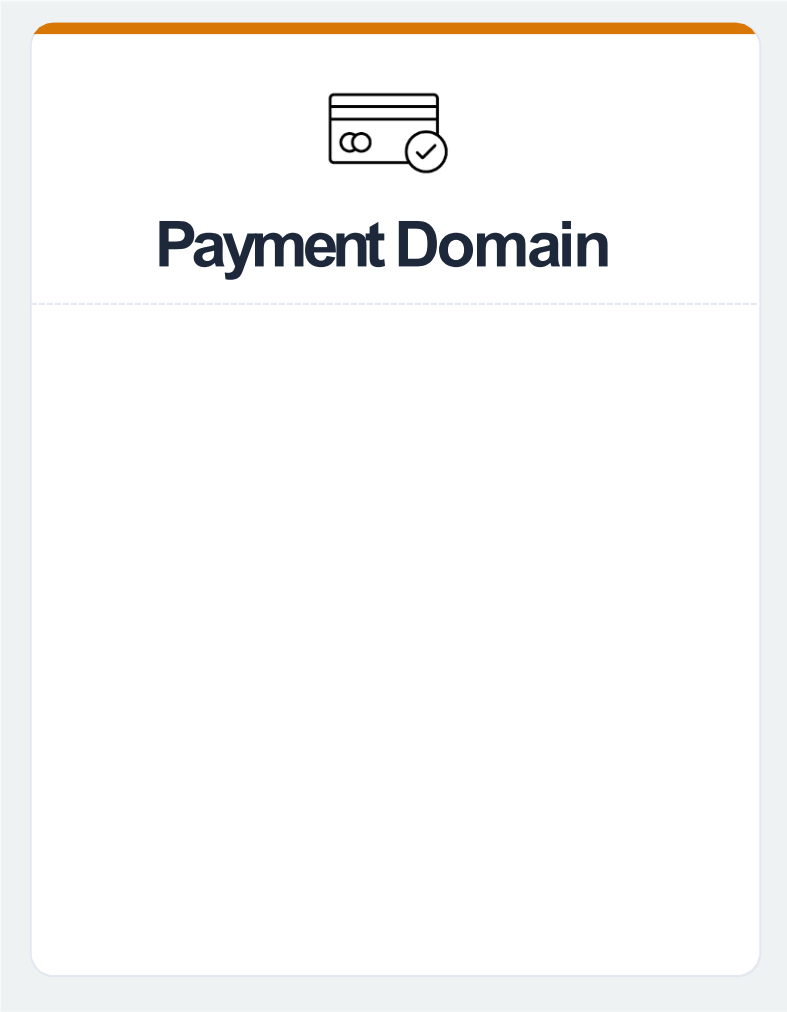
Room Domain

C4 Level 4 - Code View (Room Domain)



C4 – Code Slices

DETAILED VIEW (DOMAIN SLICES) - LEVEL 4



Technology Stack and Data Model

TECHNOLOGY & DATA STRUCTURE



INFRASTRUCTURE & CODE

Tech Stack

Backend

Flask (Python)

Main framework, monolithic REST API

Middleware

flask_cors

Secure handling of cross-origin requests

Database

mysql.connector

Native MySQL driver, config via .env file.

User Interface

Static User Interface

HTML5, CSS3, JS (Fetch API) in SRC/UI

Storage

Static Files

Image uploads (SRC/uploads) served by route.



ENTITIES AND RELATIONSHIPS

Data Model

Core

Rooms & Bookings

Rooms (type, price) and Bookings (dates, status)

Users

Users & Staff

User accounts and staff roles (RBAC)

Finance

Payments & E-Wallet

Transactions, e-wallet, refunds

Promotions

Coupons

Discount logic, time validity

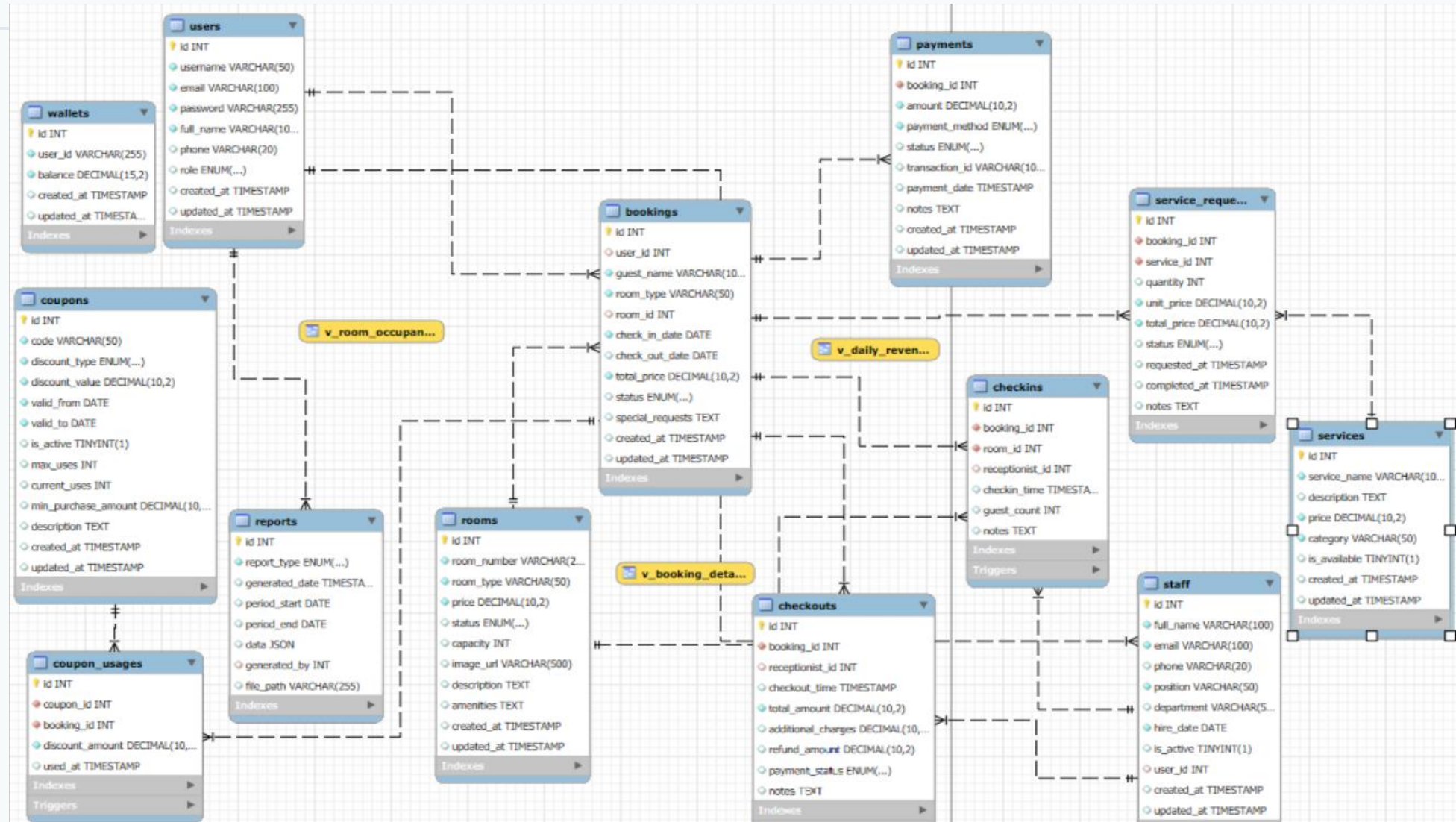
Operations

Check-in and Check-out

Operational logs of arrivals/departures

ER Diagram

SYSTEM ENTITIES AND RELATIONSHIPS



States and Transitions

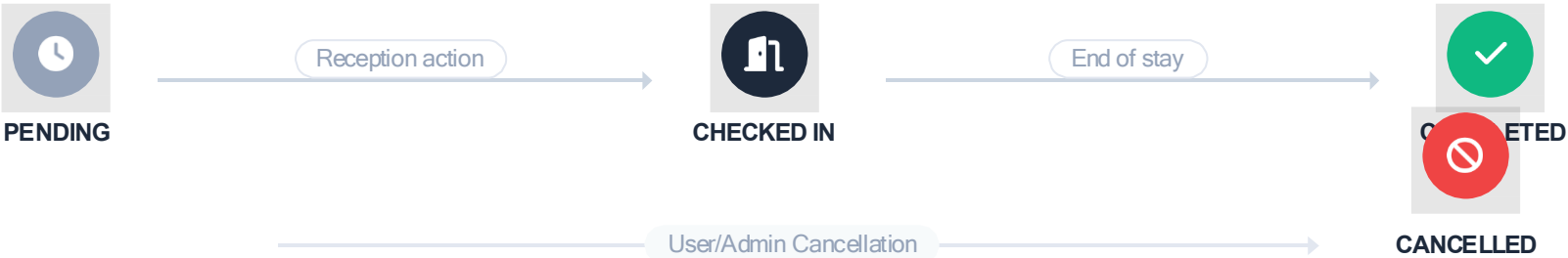
MAIN ENTITY LIFECYCLE



Booking

Lifecycle of a booking, from creation to completion or cancellation.

Rule: Cannot cancel booking after check-in.



Room

Room inventory management and real-time availability updates.

Rule: Automatically freed on check-out



Payment

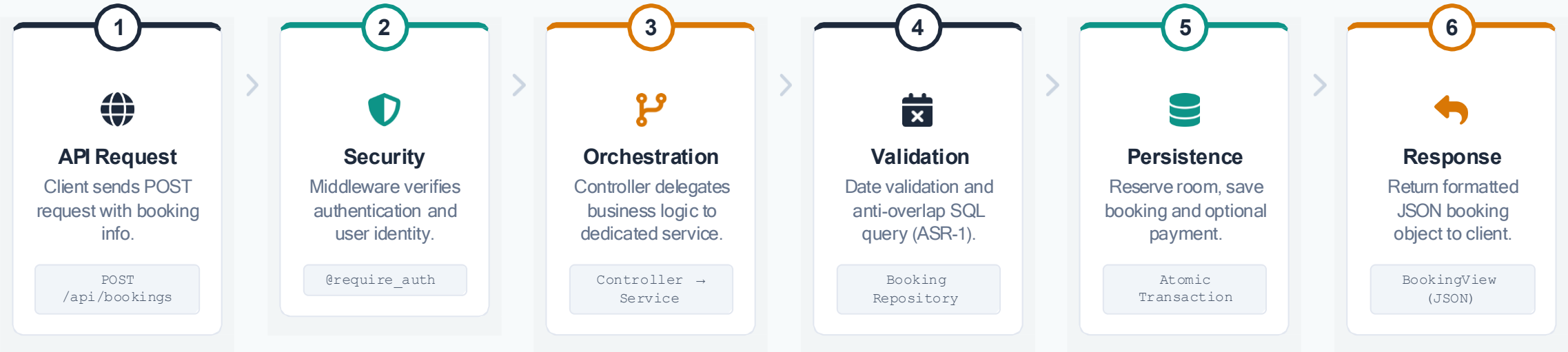
Monitoring financial transactions and refunds.

Rule: Automatic refund to user



Process

IMPLEMENTATION: CREATE BOOKING



HTTP / API Layer



Security & Data



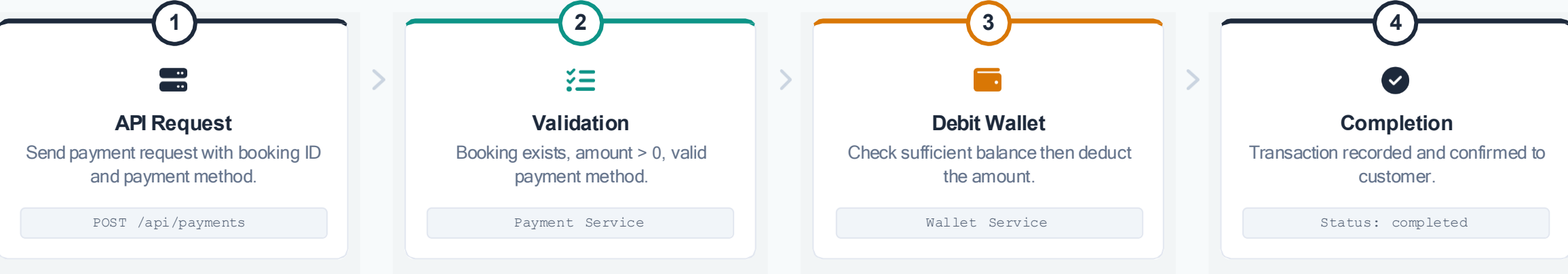
Business Logic

Key Processes: Payment & Cancellation

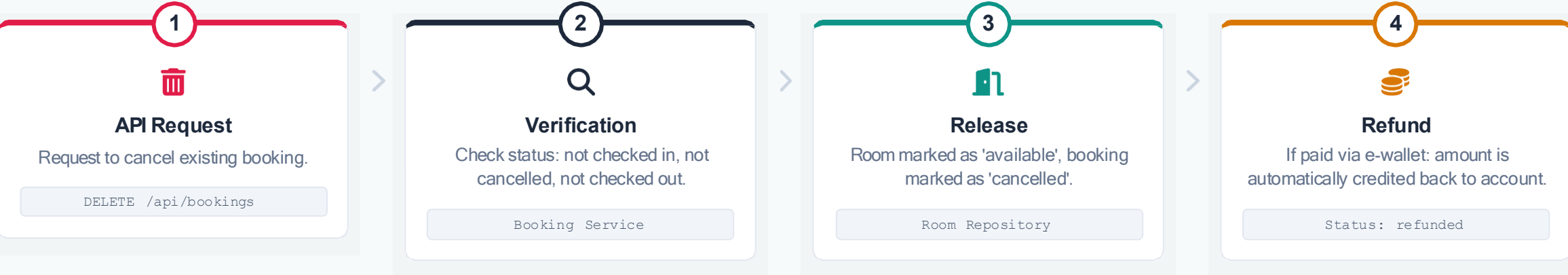
IMPLEMENTATION: FINANCIAL MANAGEMENT AND REFUNDS



Payment Flow



Cancellation and Refund Flow

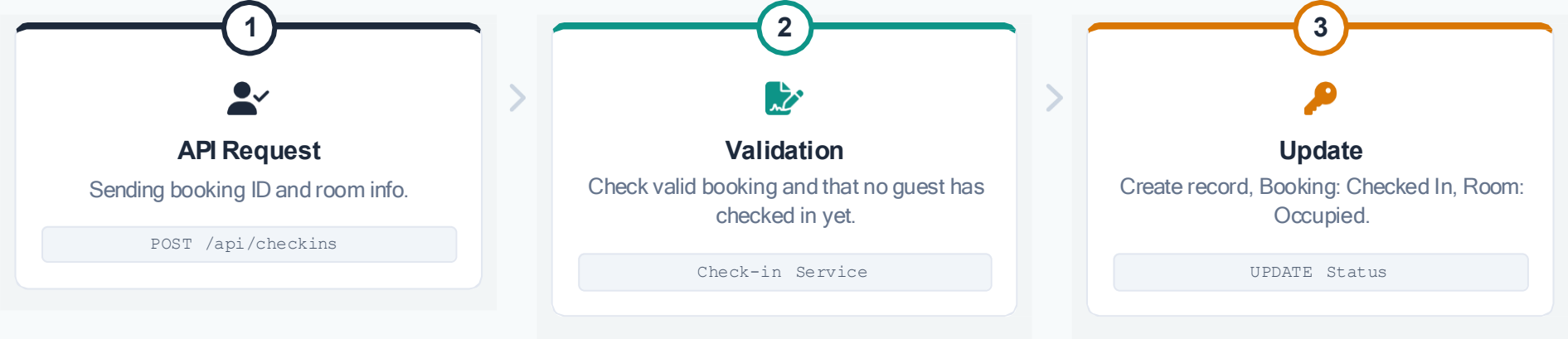


Key Processes – Check-in & Check-out

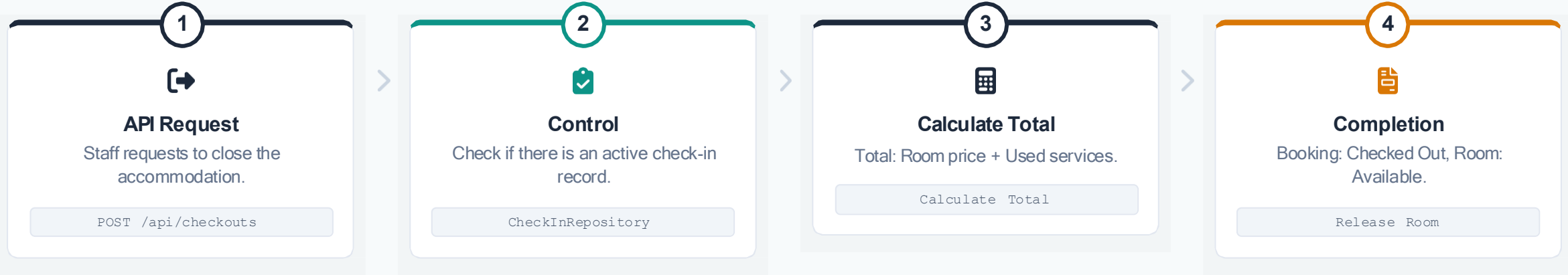
OPERATIONAL WORKFLOW: RECEPTIONIST



ARRIVAL Check-in Process



DEPARTURE Check-out Process



API Endpoint Catalog

KEY EXCERPTS BY FUNCTIONAL DOMAIN



Method	Endpoint	Auth	Authorized Roles
POST	/api/auth/login	NO	Public
GET	/api/rooms	Optional	Public
POST	/api/rooms	YES	Admin
POST	/api/bookings	YES	User Receptionist Admin
DELETE	/api/bookings/{id}	YES	Owner Receptionist
POST	/api/payments	YES	User Receptionist
GET	/api/wallet	YES	User
POST	/api/coupons/apply	YES	User

Authorization Matrix

ROLE-BASED ACCESS CONTROL (RBAC)



Role	Level	Permissions and Scope	Key Endpoints
 Public	OPEN	Anonymous access needed to enter system and load static resources.	<code>/api/auth/login</code> <code>/api/users (Register)</code> <code>/uploads/*</code>
 Guest / All	READ	Lookup public catalogs (rooms, services) without mandatory authentication.	<code>GET /api/rooms</code> <code>GET /api/services</code> <code>GET /api/coupons/{id}</code>
 User	RESTRICTED	Restricted access to personal data: bookings, e-wallet, payments.	<code>GET /api/bookings/my</code> <code>POST /api/wallet/topup</code> <code>POST /api/payments</code>
 Receptionist	OPERATIONAL	Daily management: Check-in/out, booking overview, manage service requests.	<code>/api/checkins, /api/checkouts</code> <code>GET /api/bookings (All)</code> <code>PUT /api/service-requests/{id}</code>

Testing and Verification Strategy

QUALITY ASSURANCE AND PROCESS VALIDATION



BACKEND & API

Unit and Integration Testing

UNIT	Booking Service <i>Reject past dates (Business rule validation)</i>
UNIT	Booking Repository <i>Accurate data mapping (Row → Model)</i>
UNIT	Payment Service <i>Prevent duplicate payments (Business logic)</i>
INTG	POST /api/bookings <i>Reserve room and update status in database</i>
INTG	POST /api/payments <i>Debit wallet balance and record transaction</i>



FULL FLOW

End-to-End Validation (E2E)

01	Register & Login <i>Account creation h Authentication</i>	✓
02	View rooms <i>Search, filters h details</i>	✓
03	Make Booking <i>Create booking (API POST)</i>	✓
04	Payment (Wallet) <i>Payment via internal balance</i>	✓
05	Check-in <i>Reception confirms arrival.</i>	✓

Conclusion and Lessons Learned

PROJECT SUMMARY AND TECHNICAL ANALYSIS



Achievements

OBJECTIVES ACHIEVED



Operational Multi-role System

Completed implementation of User, Receptionist, and Admin roles with dedicated workflows and distinct interfaces.



Modular Architecture (C4)

Strict separation (Controller → Service → Repository) ensures modifiability and testability.



Functional requirements met.

100% support for critical functions: Booking, Payment, Check-in/out, E-wallet, and Reporting.



Quality & Security (NFR)

ASR Validation: Anti-overbooking, financial transaction security, and RBAC.



Lessons Learned

OBSERVATIONS



Value of Strict Layering

This clear separation allows isolating business logic (Service) from data access, simplifying maintenance.



Data Consistency

Transaction management is key for the booking/payment process to avoid inconsistencies.



Security via Middleware

Declarative approach (@require_auth) provides stronger security and is less error-prone than manual checks.



SQL Mastery

Using direct SQL queries in the repository allows better performance control (indexes, joins).

Roadmap – Future Enhancements

NEXT STEPS AND TECHNICAL DEVELOPMENT



Security

- JWT authentication and token refresh
- Account lockout after failed attempts
- Audit logging for sensitive actions



Analytics

- Enhanced financial reporting
- Automated CSV/PDF exports
- Interactive visual dashboards



Scalability

- Docker Containerization
- Cloud Deployment & Load Balancing
- Transition to Microservices architecture



User Experience

- Modern responsive interface
- Transactional Email Notifications
- Live Chat Support



Operations (Ops)

- Centralized structured logging
- Proactive Monitoring & Alerting
- API error standardization